

Cosa fanno ApiController, AuthApiController e CsrfApiController

Approfondimento sul backend Spring Boot — Tornei di Calcio

1. AuthApiController — Chi è loggato?

Questo controller espone un solo endpoint: **GET /api/auth/me**. Il suo scopo è permettere a React di sapere **se l'utente ha una sessione valida** e, in caso affermativo, chi è e che ruolo ha (admin o utente normale).

Quando il frontend si carica (o ricarica), il browser invia automaticamente il cookie **JSESSIONID** a ogni richiesta dello stesso dominio. Spring Security legge questo cookie e ricostruisce l'oggetto **Authentication** che rappresenta l'utente loggato. **AuthApiController** si limita a leggere questo oggetto e a restituirlo in formato JSON.

```
@RestController
@RequestMapping("/api/auth")
public class AuthApiController {

    @GetMapping("/me")
    public ResponseEntity<Map<String, Object>> me(Authentication authentication) {
        if (authentication == null || !authentication.isAuthenticated()
            || "anonymousUser".equals(authentication.getPrincipal())) {
            return ResponseEntity.status(401).build(); // non loggato
        }
        boolean isAdmin = authentication.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .anyMatch(ADMIN_ROLE::equals);

        Map<String, Object> body = new HashMap<>();
        body.put("username", authentication.getName());
        body.put("isAdmin", isAdmin);
        return ResponseEntity.ok(body); // es: {username: "mario", isAdmin: true}
    }
}
```

Chi lo chiama nel frontend? **AuthContext.tsx** lo chiama in un **useEffect** al caricamento dell'app. In base alla risposta (200 con dati o 401), setta lo stato globale **isAuthenticated** e **isAdmin**, che poi controllano la visibilità dei pulsanti (logout, area admin, preferiti).

2. CsrfApiController — Il token di sicurezza

Spring Security abilita di default la protezione **CSRF** (Cross-Site Request Forgery). In pratica: le richieste che **modificano** dati (POST, PUT, DELETE) devono includere un **token segreto** che il server ha generato per quella sessione. Questo impedisce che un sito malevolo possa inviare richieste a nome dell'utente loggato.

Il problema: React (il frontend) deve conoscere questo token per poter inviare richieste POST o DELETE. **CsrfApiController** espone **GET /api/csrf** che restituisce il token corrente e il nome dell'header in cui deve essere inserito.

```
@RestController
```

```

@RequestMapping("/api")
public class CsrfApiController {

    @GetMapping("/csrf")
    public Map<String, String> csrf(CsrfToken token) {
        Map<String, String> body = new HashMap<>();
        body.put("headerName", token.getHeaderName()); // "X-CSRF-TOKEN"
        body.put("token", token.getToken());           // "abc123..."
        return body;
    }
}

```

Chi lo chiama nel frontend? La **Navbar.tsx** lo chiama **prima di fare il logout**. Il logout in Spring Security richiede una POST a **/logout** con il token CSRF valido. Il frontend lo recupera da **/api/csrf**, lo mette nell'header e poi invia la richiesta di logout.

```

// Esempio nel frontend (Navbar.tsx)
const csrfRes = await api.get('/api/csrf');
const { headerName, token } = csrfRes.data;

await api.post('/logout', null, {
  headers: { [headerName]: token },
});

```

Nota: le chiamate GET (come **/api/home**) NON richiedono il token CSRF perché non modificano dati. Il token serve solo per operazioni di scrittura.

3. ApiController — Operazioni sui dati

Questo è il controller **generico** che espone le API REST per le operazioni sui dati che non rientrano nella home aggregata. Gestisce: partite di un torneo, squadre di un torneo, e preferiti dell'utente.

A differenza di **HomeApiController** (che restituisce DTO tipizzati), **ApiController** costruisce le risposte "al volo" usando **Map<String, Object>**. Questo è un modo più rapido ma meno strutturato di creare JSON: invece di definire una classe Java apposita, si compone un dizionario di chiavi/valori direttamente nel controller.

```

@RestController
@RequestMapping("/api")
public class ApiController {

    // 1. Elenco partite di un torneo (GET /api/tornei/{id}/partite)
    @GetMapping("/tornei/{id}/partite")
    public ResponseEntity<List<Map<String, Object>>> getPartite(@PathVariable Long id) { ... }

    // 2. Elenco squadre di un torneo (GET /api/tornei/{id}/squadre)
    @GetMapping("/tornei/{id}/squadre")
    public ResponseEntity<List<Map<String, Object>>> getSquadre(@PathVariable Long id) { ... }

    // 3. Aggiungi ai preferiti (POST /api/tornei/{id}/preferito)
    @PostMapping("/tornei/{id}/preferito")
    public ResponseEntity<Void> aggiungi(@PathVariable Long id) { ... }

    // 4. Rimuovi dai preferiti (DELETE /api/tornei/{id}/preferito)
    @DeleteMapping("/tornei/{id}/preferito")
    public ResponseEntity<Void> rimuovi(@PathVariable Long id) { ... }

    // 5. I tuoi preferiti (GET /api/utente/preferiti)
    @GetMapping("/utente/preferiti")
    public ResponseEntity<Set<Long>> getPreferiti() { ... }
}

```

Endpoint per partite e squadre: questi due endpoint restituiscono dati JSON usati dalle pagine Thymeleaf (o potenzialmente da React in futuro) per mostrare il calendario, il dettaglio partita e le squadre. Il controller costruisce manualmente ogni campo del JSON (id, dataOra, luogo, stato, goals, squadraCasa, squadraOspite).

Endpoint per i preferiti: questi sono gli endpoint usati DIRETTAMENTE dal frontend React quando l'utente clicca sulla stella su una card del torneo. Richiedono l'autenticazione: se l'utente non è loggato, restituiscono **401 Unauthorized**.

4. Tabella riassuntiva

Controller	Endpoint	Metodo	Chi lo chiama	A cosa serve
AuthApiController	/api/auth/me	GET	AuthContext.tsx	Verificare se l'utente è loggato e leggere ruolo
CsrfApiController	/api/csrf	GET	Navbar.tsx	Ottenere il token CSRF per POST/DELETE sic
ApiController	/api/tornei/{id}/partite	GET	Pagine Thymeleaf / React	Elenco partite di un torneo
ApiController	/api/tornei/{id}/squadre	GET	Pagine Thymeleaf / React	Elenco squadre di un torneo
ApiController	/api/tornei/{id}/preferito	POST	TorneoCard.tsx (React)	Aggiunge un torneo ai preferiti
ApiController	/api/tornei/{id}/preferito	DELETE	TorneoCard.tsx (React)	Rimuove un torneo dai preferiti
ApiController	/api/utente/preferiti	GET	torneoService.ts (React)	Recupera gli ID dei tornei preferiti

5. Perché esistono controller separati?

AuthApiController e **CsrfApiController** sono "infrastrutturali": non gestiscono dati di business (tornei, squadre), ma **stato della sessione e sicurezza**. Sono separati per chiarezza: un controller si occupa solo di autenticazione, l'altro solo del token CSRF.

ApiController invece è un controller "misto" che raccoglie operazioni API varie che non hanno un controller dedicato (come **HomeApiController** che ha un solo scopo: la home). In un progetto più grande, gli endpoint di **ApiController** potrebbero essere spostati in controller dedicati (es. **PartitaApiController**, **PreferitoApiController**).

HomeApiController restituisce DTO (Java Records) tipizzati e ben strutturati. **ApiController** invece usa **Map<String, Object>**: questo è più veloce da scrivere ma meno robusto. È una scelta pragmatica, ma in un progetto in crescita conviene uniformare tutto con DTO tipizzati anche lì.

Documento generato automaticamente dall'analisi del codice sorgente.